

A Code Copilot Built on CodeSearchNet

RAG + LoRA + Agent — A retrieval-augmented system, LoRA fine-tuning, and a code-search-and-explain agent
Applied Languages Model · Deep Learning and AI Model Development · Google & Reichman Tech School
Instructors: Dr. Barak Or, Or Zooberner · Submitted by: Tal Matzliach

Abstract — This report documents a small Python code assistant built on top of CodeSearchNet: a RAG pipeline retrieves relevant functions, an optionally LoRA-tuned generator explains them, and a minimal agent retrieves, explains, and cites the source — or abstains when there is no confident match. Across 16 LoRA configurations, two independent LLM judges, and bootstrap-confirmed significance tests, **no trained variant beat the base model** on judged answer quality, despite consistently higher word-overlap scores — so the shipped system uses the base model unchanged. Both retrieval and the full agent perform well on unseen test data (Hit@1 $\approx$ 82%, agent success $\approx$ 83%). A small paraphrase check (n=8) points to a gap between docstring-style queries and free natural-language questions, detailed in the limitations section.

1. Introduction and Motivation

The initiative to build this system grew out of wanting a day-to-day coding aid that doesn't depend on an external service, alongside a clear intent to move beyond theory and get hands-on experience implementing all three techniques taught in the course — retrieval-augmented generation (RAG), parameter-efficient fine-tuning via LoRA, and agentic workflows. For that reason, a single solution combining all three was built, rather than focusing on one technique alone.

The resulting solution is a small **Code Copilot**: given a natural-language query, the tool locates the most relevant function from a large code corpus, optionally uses a LoRA-tuned generator to explain it, and returns an answer grounded in the original code snippet it was based on — or abstains when there is no confident match.

2. Dataset and Preprocessing

The dataset used for the project is **CodeSearchNet** (Husain et al., 2019), a collection of code functions labeled with documentation (docstrings) gathered from open-source GitHub repositories. The Python subset of this corpus was used, chosen because it is widely used, well documented, and allows direct evaluation of code-explanation quality against the existing docstrings as a reference.

CodeSearchNet provides official **repository-disjoint** train / validation / test splits — a repository appearing in one split does not appear in another, which preserves a fair evaluation. The official **validation** label is kept internally as **val**; all three splits were verified after filtering to contain zero repository overlap.

To illustrate the data's structure, here is an actual row from `ds["train"][0]`:

```
Raw record · repository_name = proycon/pynlpl
func_path_in_repository: pynlpl/formats/folia.py
func_name: AbstractElement.addidsuffix · language: python
func_documentation_string:
"Appends a suffix to this element's ID, and optionally to all child
IDs as well."
func_code_string:
def addidsuffix(self, idsuffix, recursive = True):
    """
    if self.id: self.id += idsuffix
    if recursive:
        for e in self:
            try:
                e.addidsuffix(idsuffix, recursive)
            except Exception:
                pass
```

Mainly two fields from each such row are used: `func_code_string` (the function body, embedded, retrieved, and explained later) and `func_documentation_string` (the docstring, used as the retrieval query and reference answer). The remaining fields are mostly used for tracking, filtering, and citing the source.

Before training and evaluation, three quality filters were applied, in this order, across all three splits: (1) removal of empty or placeholder documentation, plus exact-duplicate removal — when the same code snippet appeared in more than one split, it was kept only in the split with the highest priority order `train`, `val`, `test`, so that no function appeared in both the training set and an evaluation set; (2) a language check that kept only English-language documentation; (3) a combined 512-token budget (doc+code), measured with the embedding model's own tokenizer.

The model `sentence-transformers/all-MiniLM-L6-v2`, used for retrieval and building the vector store, was chosen for several practical reasons: it is small and fast enough to embed the entire filtered corpus on a single Colab GPU, requires no special access permissions on Hugging Face, and is strong enough as a general-purpose semantic encoder for matching docstrings to code. Its original context length — exactly 512 tokens — is the source of the filtering budget mentioned above, so the filter matches the model that actually runs retrieval.

Comparing the full corpus before and after filtering, the token-budget filter removed the largest share among the four filters, but at a moderate intensity: out of 6,593 code repositories with 10 lines or more, the median survival rate was 88.5%, and only about 0.5% of repositories lost more than 80% of their lines. The length distribution before and after nearly overlapped, with trimming concentrated mainly in the tail of unusually long functions — so the 512-token cutoff trimmed edges rather than distorting the overall picture of the data.

Since the raw code string explicitly contains the docstring, every training and generation input is built from the alternate field `func_code_no_doc` rather than the original code — forcing the model to generate the docstring instead of copying it.

After filtering, every row in the final table is converted into a "chunk" with two parallel representations: `chunk_text` — the docstring followed by the full code (including the docstring embedded within it) — and `func_code_no_doc`, the same function without the docstring. This split serves two different purposes: `chunk_text` is embedded and displayed at retrieval time, where full readability helps make sense of the result; and `func_code_no_doc` is used as the generation input, so the model is required to produce the docstring rather than copy it. Both representations are stored together in the Chroma vector store, so that the retrieval and agent evaluations (Section 3) query the same index.

Here is that exact same row (`proycon/pynlpl`, `chunk_id=0`) after conversion into a chunk, in both representations:

```
Chunk · same row after processing
chunk_text — for retrieval and display (docstring + code together):
Appends a suffix to this element's ID...

def addidsuffix(self, idsuffix, recursive = True):
    """Appends a suffix to this element's ID..."""
    if self.id: self.id += idsuffix
    if recursive:
        for e in self:
            try:
                e.addidsuffix(idsuffix, recursive)
            except Exception:
                pass
func_code_no_doc — for generation (docstring removed):
```

```
def addidsuffix(self, idsuffix, recursive = True):
    if self.id: self.id += idsuffix
    if recursive:
        for e in self:
            try:
                e.addidsuffix(idsuffix, recursive)
            except Exception:
                pass
```

After applying all the steps above, the final chunk table totaled 347,590 rows, split into **train** 313,620 (90.2%), **val** 17,007 (4.9%), and **test** 16,963 (4.9%) — a split that roughly preserves CodeSearchNet's official split ratios, and forms the data foundation for every retrieval, training, and evaluation stage in the rest of this report.

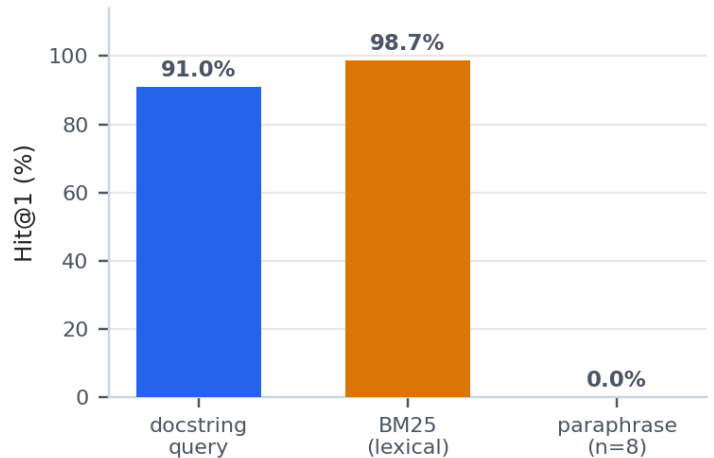
Worth noting: most checks in this report do not run on the full 347,590 chunks, but on small random samples of them — between 40 and 200 rows per check, depending on the stage. The practical reason: running full training or evaluation (mainly LLM calls for judging, or LoRA training) on hundreds of thousands of rows would have required time and compute beyond the limits of a single-GPU Colab project. Samples were drawn randomly, stay within their appropriate split (only train for training, only val for calibration, only test for final evaluation), and are distinct from one another, so there is no overlap between stages. The exact sample size is stated for each check individually; results should be read as directional and reliable in order of magnitude, not as a full-scale statistical study.

Sanity check for retrieval

Before building the full pipeline, a focused check asked whether semantic embeddings add value beyond simple word overlap, and whether docstring-style queries represent how a real user would actually ask. On a sample of 15,000 functions, the same self-retrieval task (recovering a function's code from its own query) was run three ways.

As shown in Figure 1, embedding-based retrieval against the real docstring reached 91.0% Hit@1 — a score that looks good, but turned out to be misleading: a much simpler lexical baseline (BM25, based purely on word overlap) scored even higher, 98.7%, showing that the embedding did not actually add meaningful semantic value beyond vocabulary overlap. The third check revealed the full picture: when those same eight questions were rephrased as natural language, the way a real user would actually ask, Hit@1 collapsed entirely to 0.0%.

Figure 1 — Hit@1 in retrieval, by query type



For example, the corpus contains one code chunk whose docstring is "For debugging, dump the entire data structure" (from the repository `aichaos/rivescript-python`). When using a phrasing close to that as the query, as was done in 91% of cases on average, retrieval finds it easily; but when asking about the exact same intent in natural language — "How do I sort a dictionary and get key-value pairs?" — the correct chunk ranked **4,303rd out of 15,000** candidates. This example illustrates what shared vocabulary between docstring and code conceals: when the query's phrasing is close to the original docstring, retrieval looks nearly perfect — but that same question, phrased the way a real user would actually ask it, collapses entirely. This result shaped how later numbers in this report should be read: retrieval and agent scores further on (Sections 4-5) use docstring-style queries and therefore measure a best-case scenario, as detailed in the limitations section.

3. Model Design: LoRA, RAG, and the Agent Pipeline

The system's base model, `Qwen2.5-Coder-3B-Instruct`, was chosen from an early screen of 8 candidate models across two task directions (`code`→`doc`, `doc`→`code`), scored by ROUGE-L, embedding similarity, and an LLM judge.

The selection process involved two steps. First, the task direction was checked: `code`→`doc` means giving the model code and asking it to write the documentation (docstring) describing it, while `doc`→`code` means the reverse direction — giving the model documentation and asking it to write the matching code. The `code`→`doc` direction turned out to be easier and higher-quality across all 8 models (average GPT-judge score 2.94 vs. 2.29), and was therefore chosen as the task direction.

Figure 2 — 8-model screen, n=20: GPT judge, coc

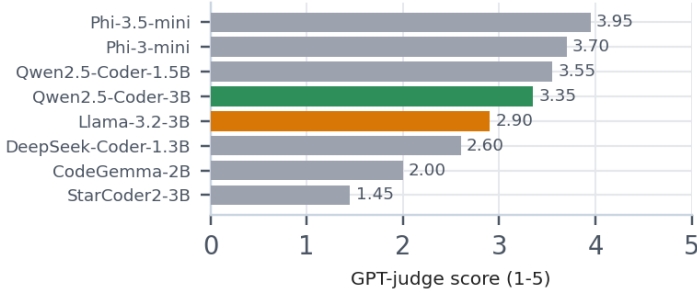


Figure 2. GPT-judge score (1-5) alone for each of the 8 models (n=20 samples per model), in the `code-doc` direction.

The Phi models led on the GPT-judge score alone (Figure 2), but that is only one of five metrics used for the decision — together with ROUGE-L, embedding similarity, syntax validity, and the judge score in the reverse `doc-code` direction, shown in Figure 3. On a composite score weighting all five metrics equally (not just the GPT judge), **Llama-3.2-3B** and **Qwen2.5-Coder-3B** tied at the top of the table (0.90 each) — so they, not Phi, advanced to the final comparison stage.

Figure 3 — remaining metrics in the 8-model screen, n=20 (ROUGE-L, embedding, syntax, judge doc-to-code)

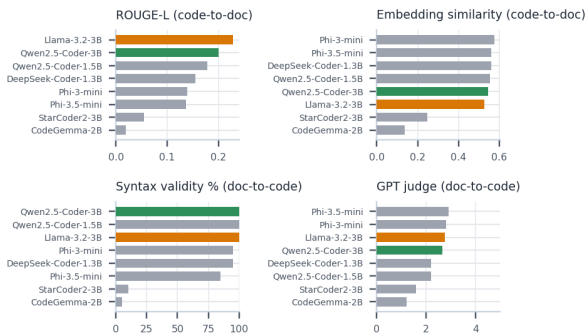


Figure 3. The four additional metrics used in the composite score, for each of the 8 models (n=20 samples per model): ROUGE-L and embedding similarity (`code-doc` direction), syntax validity and GPT-judge score (`doc-code` direction). Llama leads only on ROUGE-L; on the remaining metrics it and Qwen are close, or Qwen leads slightly.

Since the two leading candidates were so close on the composite score, the question remained whether the difference between them was real or coincidental — so a paired bootstrap test was run (5,000 resamples, n=20 and n=40), which checks whether a gap between two models holds up when the data is resampled, or is simply a result of chance. The test found that Llama's advantage was not real on most metrics: the only difference that was consistent and significant at both sample sizes was on ROUGE-L (word overlap against the true documentation) alone; on the remaining metrics — semantic similarity, syntax validity, and the judge scores in both directions — the difference was not large enough to be considered real. Since Llama's advantage turned out to be marginal, the decision fell to practical considerations: Qwen2.5-Coder-3B does not require gated access (unlike Llama on Hugging Face), and — more importantly — unlike Llama, which is a general-purpose model, Qwen2.5-Coder was trained specifically on code, so it "understands" code and also writes natural-language text about it better, for this project's specific task.

The agent's retrieval tool queries the vector store built in Section 2 (347,590 code chunks that passed the filtering stages), and returns the chunk (or chunks) closest to the query along with a similarity distance. On top of this, LoRA adapters were trained — a fine-tuning method that trains only a small additional set of weights instead of retraining the whole model — a key advantage when training on a relatively small sample of about 400 rows, as here, where full fine-tuning risked "memorizing" the examples rather than learning from them. The adapters were restricted to the attention layers only (`q_proj`, `k_proj`, `v_proj`, `o_proj`) rather than the whole model, to reduce the number of learned parameters and lower the risk of memorization on a small sample. The learning rate (1.5e-4) and number of epochs (up to 2) were chosen conservatively for a similar reason — too little training and the model won't learn enough, too much on only 400 rows and it risks memorizing them instead of generalizing. The loss was also computed only on the response (docstring) and not on the prompt the model sees as input, so that the score reflects only the quality of the model's writing and not its ability to reproduce text it already received. An initial comparison between **r=8** and **r=16** (adapter size) revealed an unexpected split — ROUGE-L (word overlap) improved, but the LLM-judge score (perceived quality) dropped — meaning the model "sounded" closer to the correct answer in wording, but in practice wrote worse answers. This gap between the two kinds of measurement was the main suspect that something was off in the process, and prompted an extended test of 16 configurations (Section 4) to understand its source.

Above retrieval and LoRA tuning sits the agent itself, deliberately kept small: retrieve ← optional keyword-filter tool ← generate explanation ← cite the source. The critical step is the confidence gate: the distance between the query and the best retrieval result is measured, and if it is too large (above a threshold pre-calibrated on **val** data) the agent answers "I don't know" instead of responding from a weak match. The idea behind this choice is that a confident but wrong answer is worse than an honest admission that there is no answer — in other words, it is better for the agent to know when it "doesn't know" than to guess and explain code that has nothing to do with the question asked. It was deliberately decided, given the project's time and resource constraints, to invest in the retrieval store and model tuning rather than building a complex agent that plans multiple steps on its own. This keeps the agent's behavior simple, predictable, and easy to explain.

4. Training Process and Metrics

Because the LoRA-vs-base question showed conflicting signals from the very first run, the investigation was expanded to **16 trained configurations** in total. Thirteen of them tested systematic variations on the same training recipe: adapter rank (**r**) — the size of the learned adapter, where a higher rank lets the model learn more complex changes but also increases the risk of memorization on a small sample; learning rate (how large each update step is — too high risks unstable training, too low won't learn enough) and number of epochs (how many times the model passes over the data — too many increases memorization); full fine-tuning (updating all model weights) vs. LoRA (updating only a small addition); and four versions of how **LoRA's own training data** was built (a further filter applied only to the training subsample, distinct from the general corpus in Section 2) — unfiltered (the original 400 rows as-is), detail-filtered (only examples with documentation detailed enough, at least 15 words, to rule out documentation that is too short and sparse), and stricter-filtered on 3× the data (the same filter but stricter

— at least 30 words — over a sample three times larger), plus checkpoint (choosing the best stopping point during training) and regularization variants (techniques that reduce memorization, such as weight decay and dropout). Each such change was meant to test whether the problem stemmed from the sample size, the adapter itself, or the data quality.

The two additional runs tested entirely different questions: a run with 10× more training data tested whether the problem disappears at larger scale, and a pilot that rewrote the training targets using GPT-4o tested whether the quality of the original docstrings (rather than the training method) was the cause. Every configuration was measured on the same metrics mentioned above, together with an additional LLM judge (Gemini, alongside GPT) to reduce the bias of relying on a single judge. The bootstrap test was repeated across all 16 configurations, providing each with a 95% confidence interval — that is, not just whether the drop from the base model is real, but also how large it is.

Figure 4 — LoRA sweep (overlap up, quality down), 14 runs, n=70

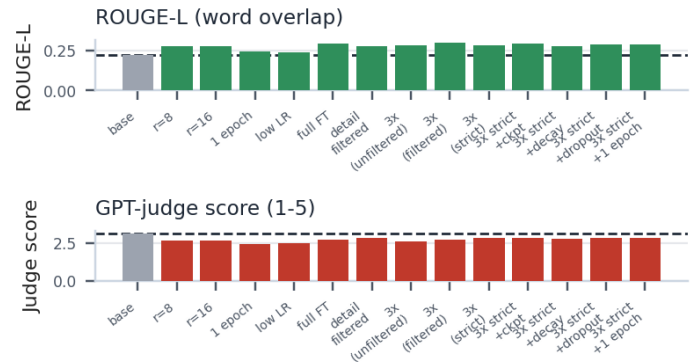


Figure 4. The base model (gray) against all 13 runs of the main sweep, on two metrics (n=70 rows from the val split per run).

Top: ROUGE-L rises above the base model's dashed line in almost every trained run — LoRA improves literal wording overlap against the reference docstring.

Bottom: GPT-judge score falls below the base model's dashed line in every trained run, without exception. The two panels tell opposite stories about the same 13 models.

Figure 5 — all 16 variants below base, n=70

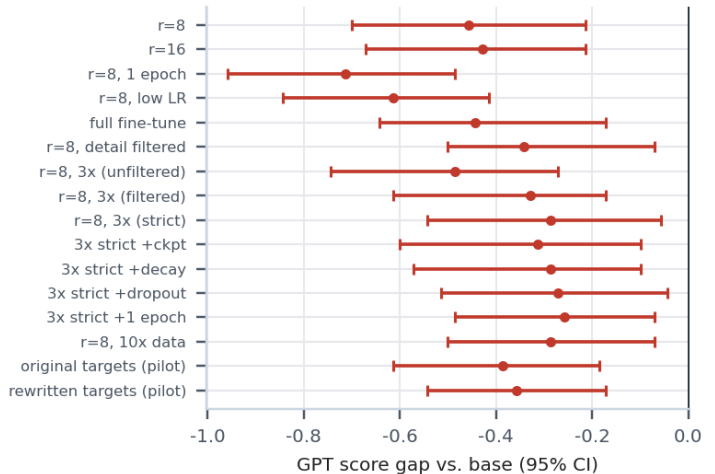


Figure 5. Each row in the chart is one trained configuration (n=70 rows from the val split per configuration). The dot shows how much lower its score is than the base model's score (0 = identical to base; the farther left the dot, the larger the gap from base). The line around the dot, representing the confidence interval, shows a plausible range for the true result — and when even the right-hand end of the line does not reach zero, that is a sign the drop is not coincidental but real. Across all 16 configurations the line sits entirely to the left of zero, meaning all 16, without exception, are significantly below the base model. Even the least-bad version, **LoRA r=8, 3× strict + 1 epoch**, still loses to the base by a gap of -0.257.

Three of the configurations were also checked with a second, independent judge (Gemini), to confirm the drop was not caused by a single judge's bias. The configurations were the 10× data-expansion run, and a pair of controlled runs (one trained on the original documentation, the other on the exact same documentation after being rewritten, so the only difference between them is the wording quality of the targets). Gemini confirmed the same direction in two of the three, with a drop of 0.27 points (10× data) and 0.49 points (original documentation) below the base. In the rewritten-documentation run the drop was significantly smaller, only 0.04 points, close to zero. In other words, two different, independent judges reached the same overall conclusion — that the trained model falls below the base — though not always with the same intensity.

Based on these results, the strongest candidate — **LoRA r=8, 3× data (detail-filtered)** — was chosen for a follow-up check against the base model on the **TEST** split, to confirm the results hold consistently on data the model had never been exposed to before (Section 5).

5. Results, Analysis, and Limitations

Final Check: LoRA vs. Base on the Test Set

The strongest candidate, **LoRA r=8, 3× data (detail-filtered)**, was checked and compared against the base model on 40 examples from the **TEST** split. The result reproduced exactly the same pattern observed on the validation set: ROUGE-L favored LoRA (0.300 vs. 0.211), and so did semantic similarity, slightly (0.640 vs. 0.633). Both LLM judges, however, favored the base model instead — GPT: 3.175 vs. 3.050; Gemini: 4.100 vs. 3.850. In other words, the automatic metrics continue

to show LoRA as "closer" to the reference answer, but both judges, which are meant to reflect human-style answer quality, prefer the base — the same split already seen in Section 4, now confirmed on data the model had never seen before.

Retrieval Quality: Validation Set vs. Test Set

Retrieval in this section was checked against the agent's actual vector store (the one built in Section 2, holding all 347,590 code chunks) — unlike the check in Section 3, which used a small sample of 15,000 functions as a quick feasibility check. Two metrics were measured: Hit@k, the frequency with which the correct result appears among the top k results (k=1, 5, 10), and MRR, a measure of the correct result's average ranking (closer to 1 = closer to the top of the list). The measurement was taken twice: on the validation set, to calibrate the agent's confidence threshold (the cutoff above which it abstains instead of answering), and then on the test set, to confirm that threshold also works well on data that was not involved in calibration at all. The results were nearly identical between the two sets (Figure 6), meaning the threshold was not overfit to the validation set.

Figure 6 — retrieval: validation vs. test, n=150

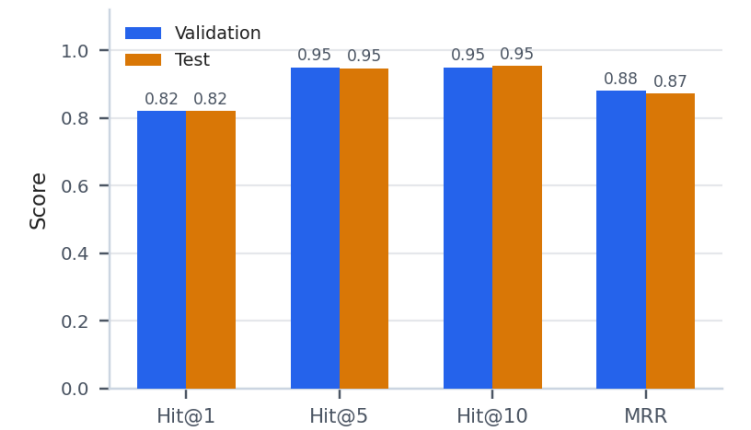


Figure 6. Each pair of bars compares retrieval quality on the validation set (blue) against the test set (orange), across four metrics (n=150 queries per set): Hit@1, Hit@5, Hit@10 (the frequency with which the correct result appears among the top k results), and MRR (the correct result's average rank position). Scores are nearly identical between the two sets on every metric — a sign that the confidence threshold calibrated on validation also fits the test set well, which the model was never exposed to at all during calibration.

End-to-End Agent Quality

The retrieval check in Figure 6, performed in isolation and checking only whether the retrieval component finds the correct result, did not examine the rest of the agent's chain. The following check, by contrast, runs the full agent end to end: retrieving the chunk best matching the query, checking it against the confidence gate (the distance between the query and the result, against the threshold calibrated in the previous section), and finally answering with a source citation — or explicitly abstaining if the match is too weak. This check therefore measures what actually matters to the end user: whether the agent, end to end, gave a good answer or wisely abstained.

A batch (a group of examples checked together) of 60 chunks from the test split was sampled, and each chunk's docstring was used as a query passed through the full agent. Two independent questions are checked for each example: did retrieval find the correct code, and did the agent choose to answer or abstain — a combination that yields four possible outcomes: **hit & answered** (the desired case — the agent found the correct code and also responded), **hit & abstained** (found the right code but still declined to answer — over-caution), **miss & answered** (did not find the correct code but answered anyway — the riskiest scenario, since the agent confidently explains the wrong code), and **miss & abstained** (did not find it and correctly chose not to answer).

Figure 7 — agent results, test n=60 (success=83%)

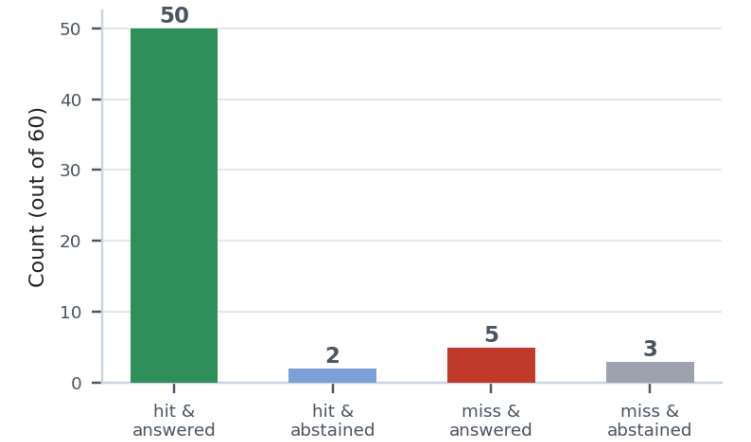


Figure 7. Agent results on the test split, n=60. 50/60 (83%) both retrieved the correct chunk and answered — slightly above the same check on validation (78%). Only 5 cases (8%) fall into the risky "miss & answered" scenario, and 3 cases (5%) correctly abstained after a miss.

Outcome	Count	Percent
Hit & answered	50	83.3%
Hit & abstained	2	3.3%
Miss & answered	5	8.3%
Miss & abstained	3	5.0%

The Agent in Action: Worked Examples

Example 1 · Grounded answer (a real docstring as the query)

Question:

"Call IUIAutomationDockPattern::SetDockPosition.
dockPosition: int, a value in class `DockPosition`.
waitTime: float.
Refer https://docs.microsoft.com/en-us/windows/desktop/api/uiautomationclient/nf-uiautomationclient-iuiautomationdockpattern-setdockposition"

Answer returned (similarity distance 0.184, below the 0.916 threshold):

```
```python
SetDockPosition(self, dockPosition: int, waitTime: float = OPERATION_WAIT_TIME) -> int:
 """
 Sets the position of the dock and waits for the operation to complete.

 Parameters:
 dockPosition (int): The desired position of the dock.
 waitTime (float): The time to wait after setting the dock position in seconds. Defaults to OPERATION_WAIT_TIME.

 Returns:
 int: The result of the operation.
 """
 ...
```

Source: yinkaisheng/Python-UIAutomation-for-Windows (uiautomation/uiautomation.py) · split=val · chunk\_id=315175

Example 2 · Justified abstention (a natural-language question)

Question:

"How does this repository validate its input?" (scoped to the repository numenta/nupic)

Answer:

The agent abstained — the closest candidate found (distance 1.341) was far beyond the confidence threshold, so it returned "no good enough match" instead of guessing.

When a query is phrased close to the real docstring (Example 1), retrieval is precise and the answer is grounded; when asking in general, natural language (Example 2), retrieval weakens and the agent abstains rather than answering from a weak match — exactly the behavior the confidence threshold was designed to produce.

Summary

The project delivered a small, functioning code copilot: on CodeSearchNet (Python), a filtered corpus of 347,590 code chunks, an embedding-based retrieval engine that finds the correct function in about 82% of cases at rank one and in 95% within the top five (Hit@1/Hit@5, n=150 queries from the test split), and an agent that answers correctly (or carefully abstains) in about 83% of cases (n=60 queries from the test split). Across all 16 trained configurations, two independent LLM judges, and bootstrap-confirmed confidence intervals, the base model **Qwen2.5-Coder-3B-Instruct** — with no LoRA adapter at all — produced the highest answer quality by the judges' scores, while the trained variants showed a consistent improvement only in the word-overlap metric (ROUGE-L). The check of the strongest candidate on TEST independently reproduced the same pattern. The implication: the off-the-shelf model is already better suited to the task, and further investment in LoRA-style tuning does not improve the current system.

Limitations

- Docstring queries, used throughout all Hit@k checks, are closer in phrasing to code than a free natural-language question (Figure 1), so real-world retrieval performance against actual users' questions is likely lower than the numbers reported here suggest.
- GPT and Gemini agreed on the direction of the quality drop, but a manual review of the 8 largest score declines (out of the 13-run sweep, scored on 70 validation rows) found agreement on specific answers in only 38% of cases — a single judge's score should not be relied on for evaluating one individual answer; its reliability is only at the level of the overall trend.
- Sample sizes were tailored to the constraints of a single GPU on Colab: 1,200 rows from train for training, 40 rows from test for scoring, out of 313,620 and 16,963 available. Results are a directional signal, not a large-scale statistical study.
- The target-rewrite pilot (GPT-4o rewrote code documentation used for training, checked on 150 examples before training) narrowed the gap for one judge (Gemini: from -0.486 to -0.043) but not the other (GPT) — it is possible the documentation itself is the cause, or that the difference between the judges is misleading.
- The agent answers "I don't know" for any weak retrieval, even when the base model itself may hold relevant knowledge.

6. Future Directions

A few natural directions emerge from this project for continued work. First, finding techniques that improve the system's ability to handle real natural-language questions, not just docstring-style queries. Second, minimizing the cases where the agent answers "I don't know" when retrieval fails to find a good match, so as to provide a grounded answer in every possible scenario. And finally, it would be worth continuing the target-rewrite pilot with a larger run, exploring a larger or differently constructed training set, and extending the agent so it can use additional tools before responding.

7. Dataset Source

Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., & Brockschmidt, M. (2019). *CodeSearchNet challenge: Evaluating the state of semantic code search*. arXiv preprint arXiv:1909.09436.  
The dataset is available on Hugging Face: [huggingface.co/datasets/code-search-net/code\\_search\\_net](https://huggingface.co/datasets/code-search-net/code_search_net), and in the official GitHub repository: [github.com/github/CodeSearchNet](https://github.com/github/CodeSearchNet).